Ejercicio para la PLF: un alquiler de vehículos

> Clase: 5AKIF
> Entrenamiento para la PLF (prueba práctica) del miércoles, 4 de noviembre de 2026
> Tiempo de trabajo en la PLF: 3 UE (horas lectivas)

Indicaciones sobre este ejercicio

Este ejercicio le muestra cómo son el enunciado y el programa de prueba en la PLF. La PLF tiene la misma estructura: un modelo de clases, una descripción de las clases y un programa de prueba que asigna puntos.

Descargue el archivo *CarRentalService.7z* de los ejercicios del capítulo *Types*. Descomprima el archivo y abra el archivo *CarRentalService.sln*.

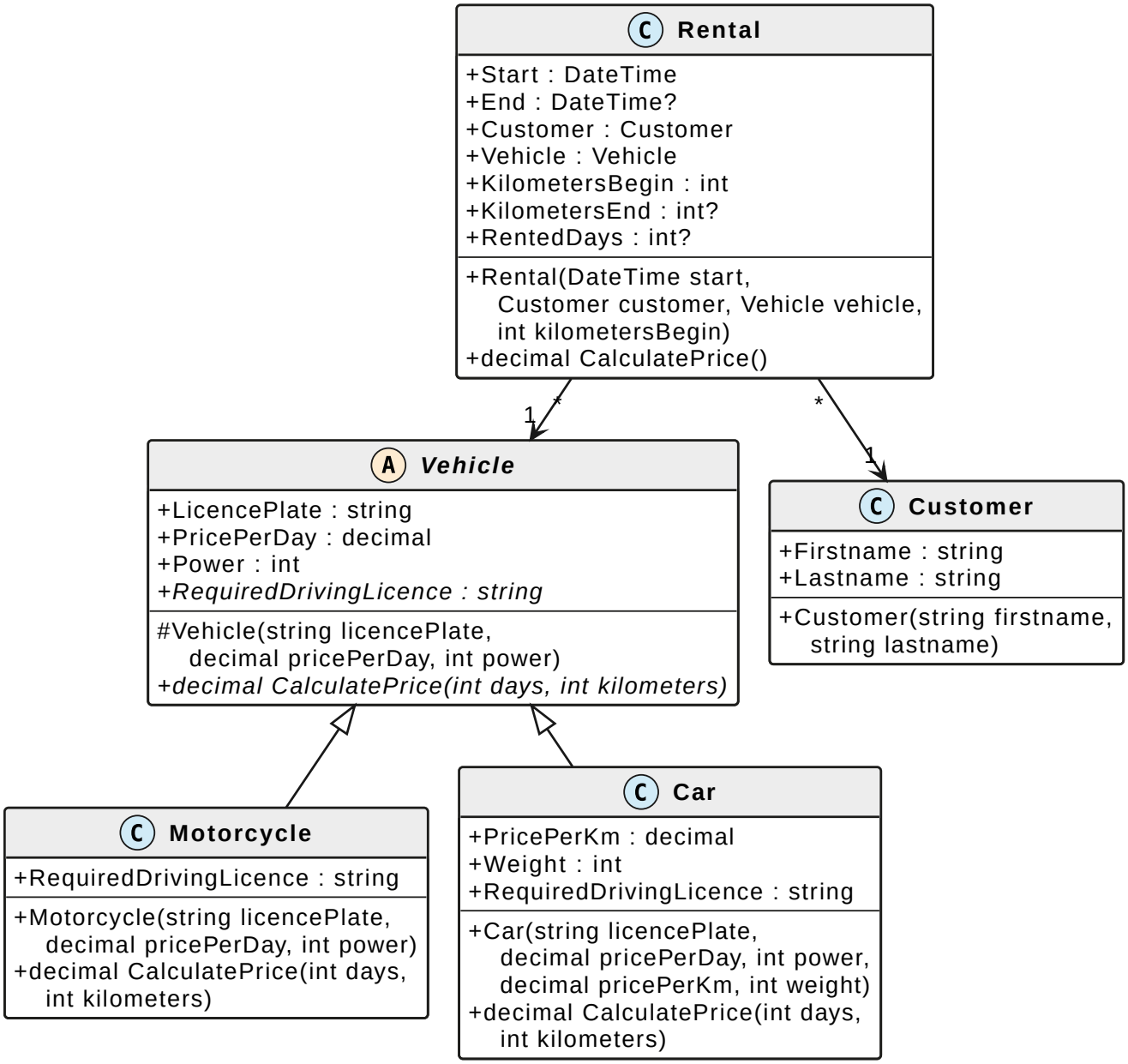


En la PLF encontrará la solution en la unidad *Z*. Allí primero escriba su nombre en el archivo *README.md*. En este ejercicio, en cambio, el archivo *README.md* es el enunciado y no tiene que escribir nada.

Tarea: un alquiler de vehículos

Una empresa de alquiler alquila motos y coches. Por una moto se paga un precio por día. Por un coche se paga además cada kilómetro recorrido. Para cada alquiler, el sistema guarda un registro de alquiler (*Rental*). Un alquiler está abierto hasta que se devuelve el vehículo.

Modelo de clases



Descripción de las clases

Cree usted mismo los campos privados que necesite para la implementación.

Clase Vehicle

La clase base abstracta para coches y motos. Todas las properties son de solo lectura (read-only).

- **LicencePlate** es la matrícula del vehículo.
- **PricePerDay** es el precio del alquiler por cada día empezado.
- **Power** es la potencia del vehículo en kW.
- El constructor es *protected*.
- **RequiredDrivingLicence** es una property abstracta. Devuelve la categoría del permiso de

conducir que se necesita para el vehículo.

- **CalculatePrice()** es un método abstracto. Calcula el precio a partir de los días de alquiler (*days*) y los kilómetros recorridos (*kilometers*).

Clase Motorcycle

Todas las properties son de solo lectura.

- **RequiredDrivingLicence** siempre devuelve *"A"*.
- **CalculatePrice()** calcula el precio con la fórmula *days* × *PricePerDay*. Los kilómetros no influyen.

Clase Car

Todas las properties son de solo lectura.

- **PricePerKm** es el precio por kilómetro recorrido.
- **Weight** es el peso en kg.
- **RequiredDrivingLicence** devuelve *"B"* si el coche pesa menos de 3500 kg. En otro caso devuelve *"C"* (camión).
- **CalculatePrice()** calcula el precio con la fórmula *days* × *PricePerDay* + *kilometers* × *PricePerKm*.

Clase Customer

La clase solo guarda el nombre y el apellido del cliente. El constructor asigna todas las properties. Todas las properties son de solo lectura.

Clase Rental

La clase guarda un alquiler.

- **Start**, **Customer**, **Vehicle** y **KilometersBegin** se asignan en el constructor y son de solo lectura.
- **Start** es el inicio del alquiler.
- **KilometersBegin** es el kilometraje al inicio del alquiler.
- **End** es el final del alquiler. Es *null* mientras no se haya devuelto el vehículo. En la devolución se asigna desde fuera de la clase.
- **KilometersEnd** es el kilometraje al final del alquiler. Es *null* mientras no se haya devuelto el vehículo. En la devolución se asigna desde fuera de la clase.
- **RentedDays** es el número de días empezados y es de solo lectura. Si *End* es *null*, *RentedDays* también es *null*. Un alquiler que termina el mismo día dura 1 día.
- **CalculatePrice()** calcula el precio del alquiler:
  - Si *End* es *null*, el método devuelve 0.
  - Si *KilometersEnd* es *null*, el método devuelve 0.

    - En otro caso devuelve el resultado de *CalculatePrice()* del vehículo alquilado. Pase como argumentos *RentedDays* y los kilómetros recorridos (*KilometersEnd* − *KilometersBegin*).

Ejemplo: Se alquila un coche con *PricePerDay* de 10 euros y *PricePerKm* de 5 euros del 10/11/2021 al 15/11/2021. El kilometraje es de 100 km al inicio y de 200 km al final.

- *RentedDays* es 6 (10, 11, 12, 13, 14 y 15 de noviembre).
- *CalculatePrice()* devuelve 6 × 10 + 100 × 5 = 560 euros.



- Si resta 2 valores *DateTime*, obtiene un *TimeSpan*. Los días empezados se obtienen con esta expresión:

```
(int)(end - start).TotalDays + 1
```

- Esto solo funciona con *DateTime*, no con *DateTime?*. Por eso, compruebe primero si *End* es *null*. Después obtiene el valor con `End.Value`. Lo mismo vale para *KilometersEnd*.
- Un literal *decimal* se escribe con *M* al final, p. ej. `0.5M`.

Trabajos a realizar

Cree cada clase en un archivo propio. Los archivos *Program.cs* y *TestHelpers.cs* **no** se pueden modificar.

Program.cs es un programa de prueba. Solo compila cuando existen todas las clases con todas las properties y todos los métodos del modelo. Preste atención también a los nombres de los parámetros de los constructores, porque el programa de prueba los usa (p. ej. `pricePerDay: 10`). Por eso, cree primero todas las clases. Los métodos y properties que todavía no ha implementado lanzan una *NotImplementedException*. Después inicie el programa en el proyecto *CarRentalService.Application*. El programa muestra *OK* o *FAILED* en cada prueba. Si una prueba lanza una excepción, la prueba no se supera. Una excepción fuera de una prueba (p. ej. al crear los datos de prueba) termina el programa.

Una solución completa muestra *OK* en las 43 pruebas. Al final, el programa muestra los puntos obtenidos y la nota:

```
Result of the program in C:\...\CarRentalService.Application:
43 of 43 checks passed.
48 of 48 points. Grade: Sehr gut.
```

Evaluación

Para obtener una nota positiva, el programa tiene que compilar. Los programas que no compilan o que se cierran con un error reciben la nota *Nicht genügend* (insuficiente). La tabla muestra todas las pruebas del programa de prueba. Los textos son los mismos que aparecen en la pantalla. Cada

prueba con *OK* da los puntos de la columna *P*.

N.º	Prueba (48 puntos en total)	P
Checking class implementation		
1	Rental does not have a default constructor.	1
2	Rental.Start is read-only.	1
3	Rental.Customer is read-only.	1
4	Rental.Vehicle is read-only.	1
5	Rental.KilometersBegin is read-only.	1
6	Rental.RentedDays is read-only.	1
7	Rental.End, KilometersEnd and RentedDays are nullable.	1
8	Customer does not have a default constructor.	1
9	Customer is immutable.	1
10	Vehicle does not have a default constructor.	1
11	Vehicle is abstract.	1
12	Vehicle has a protected constructor.	1
13	Vehicle is immutable.	1
14	Vehicle.PricePerDay is decimal and Vehicle.Power is int.	1
15	Vehicle.RequiredDrivingLicence is abstract.	1
16	Vehicle.CalculatePrice() is abstract.	1
17	Motorcycle does not have a default constructor.	1
18	Motorcycle is immutable.	1
19	Car does not have a default constructor.	1
20	Car is immutable.	1
21	Car.PricePerKm is decimal and Car.Weight is int.	1
Checking constructors		
22	Constructor of Customer sets the values.	1
23	Constructor of Motorcycle sets the values.	1
24	Constructor of Car sets the values.	1
25	Constructor of Rental sets the values.	1
Checking Motorcycle		
26	The required driving licence for motorcycles is A.	1
27	Motorcycle.CalculatePrice() returns days x PricePerDay.	2
Checking Car		
28	The required driving licence for cars < 3500 kg is B.	1

N.º	Prueba (48 puntos en total)	P
29	Car.CalculatePrice() returns days x PricePerDay + km x PricePerKm.	2
30	The required driving licence for cars >= 3500 kg is C.	1
Checking Rental		
31	The default value of Rental.KilometersEnd is null.	1
32	The default value of Rental.End is null.	1
33	Without an end date Rental.RentedDays is null.	1
34	Without End and KilometersEnd Rental.CalculatePrice() returns 0.	1
35	Rental.RentedDays returns the number of days started.	2
36	Without KilometersEnd Rental.CalculatePrice() returns 0.	1
37	Rental.CalculatePrice() uses the driven kilometers.	2
38	Without End Rental.CalculatePrice() returns 0.	1
39	Rental.RentedDays is 1 if the vehicle is returned on the same day.	1
40	Rental.RentedDays works across the end of a month.	1
41	Rental.RentedDays works across the end of a year.	1
42	Rental.CalculatePrice() charges the days if no kilometers were driven.	1
43	Rental.CalculatePrice() uses the price calculation of the rented vehicle.	2

48 - 42 puntos: Sehr gut (1, sobresaliente), 41 - 36 puntos: Gut (2, notable), 35 - 30 puntos: Befriedigend (3, bien), 29 - 24 puntos: Genügend (4, suficiente), 23 - 0 puntos: Nicht genügend (5, insuficiente). El programa muestra el nombre de la nota en alemán.